SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 3 – FUNCTION CALLING & STRUCTURED OUTPUT TASK

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO2 – Precision (BL2, BL3)	**Assessment:**	Assessment Tool 3 – Function Calling & Structured Output Task
Weightage:	40% of CIA	**Total Marks:**	1 * 100 = 100 (1 Question1)
CO2 Statement:	*Apply prompt engineering techniques and utilize pre-trained Large Language Models through modern AI frameworks and APIs to solve real-world problems (BL2, BL3)*		
Student Name:	S.Sirivally	**Reg. No.:**	192472371
Section / Batch:	2024-CSE(AI)	**Date:**	4-08-2026

Instructions to Students

- Answer 1 question. Total: 100 Marks.
- Analyze the given scenario and identify the appropriate function(s), tool(s), parameters, and structured output required to solve the task.
- Design a valid function-calling schema with appropriate function names, parameters, data types, required fields, and descriptions based on the given requirements.
- Implement the function-calling workflow demonstrating the complete process: User Query → LLM → Function/Tool Call → Function Execution → Structured Response.
- Ensure that the generated output strictly follows the defined structured format or JSON schema and contains valid, relevant, and consistent information.
- Handle appropriate edge cases such as missing parameters, invalid inputs, ambiguous requests, incorrect data types, or unavailable information, and provide suitable responses without producing invalid function calls.
- Demonstrate the working implementation using suitable test cases, including normal inputs and at least one edge-case scenario.
- Clearly document the designed schema by explaining the purpose of each function, its parameters, data types, required fields, expected input, and expected output.
- Briefly justify the design decisions and explain how function calling and structured output improve the reliability, consistency, and usability of the LLM-based application.

Submission Requirements

Students should submit:

- **Source code / Jupyter Notebook**
- **Function-calling schema**
- **Structured output/JSON schema**
- **Execution screenshots**
- **Test cases and results**
- **Brief documentation explaining the schema and function-calling workflow**

Code of Conduct

I _S.Sirivally / 192472371_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: __Sirivally.S___

38. Prompt Best-Practice Evaluation

An AI assistant uses the following prompt:

> "Tell me everything about this product and give the customer a good answer."

Task: Identify the weaknesses in the prompt and redesign it using role, context, task instructions, constraints, and a specified output format. Explain how each modification improves the prompt.

Also justify the technologies and techniques selected for the proposed system.

AI Product Information Assistant using Function Calling and Structured JSON Output

Introduction

Large Language Models (LLMs) have significantly improved the way users interact with artificial intelligence by enabling natural language conversations. However, LLMs alone cannot always retrieve accurate or up-to-date information from external systems. To overcome this limitation, Function Calling allows an LLM to invoke predefined functions or tools that perform specific tasks such as retrieving product details, checking databases, or accessing external services.

Structured Output enables the LLM to return information in a predefined format such as JSON, ensuring that responses are consistent, machine-readable, and easy to integrate into software applications. This reduces ambiguity and improves the reliability of AI systems.

In this assessment, a Python-based Product Information Assistant is developed to simulate the Function Calling workflow. The system accepts a customer query, analyzes the request, invokes the appropriate function, retrieves product information from a simulated database, and returns the response in a structured JSON format. The implementation also demonstrates validation of invalid inputs and unavailable products through appropriate error handling.

Problem Statement

Given Prompt

"Tell me everything about this product and give the customer a good answer."

Objective

To analyze the weaknesses of the given prompt and redesign it using Prompt Engineering best practices. The improved solution should incorporate role definition, context, task instructions, constraints, and a structured output format. The solution should also demonstrate Function Calling, Structured JSON Output, appropriate error handling, and multiple test cases.

Scope

The proposed system focuses on retrieving product information from a predefined product database. It demonstrates the interaction between an LLM and external functions without relying on cloud APIs. The system is implemented entirely in Python and is suitable for educational demonstration purposes.

Expected Outcome

The developed system should:

- Accept customer product queries.
- Analyze user intent.
- Select and execute the appropriate function.
- Retrieve product details from the database.
- Return structured JSON responses.
- Handle missing inputs and unavailable products gracefully.
- Demonstrate a complete Function Calling workflow.

Prompt Best-Practice Evaluation

Existing Prompt

"Tell me everything about this product and give the customer a good answer."

Weaknesses

The existing prompt has several limitations:

1. **No Role Definition**

The prompt does not specify the role of the AI assistant. Without a defined role, the generated response may vary in style and content.

2. **No Context**

The prompt provides no information about the customer, the product, or the expected audience.

3. **Unclear Task**

The phrase "tell me everything" is vague and may lead to unnecessarily long or irrelevant responses.

4. **No Constraints**

The prompt does not define response length, required information, or formatting guidelines.

5. **No Output Format**

There is no structured output requirement such as JSON or tables, making the response difficult to integrate into applications.

6. **No Error Handling**

The prompt does not specify how to respond when the requested product is unavailable.

7. **Poor Consistency**

Different executions of the prompt may generate inconsistent responses.

Redesigned Prompt

Improved Prompt

Role:

You are an AI Product Information Assistant for an online shopping platform.

Context:

Customers request information about products before making purchasing decisions.

Task:

Retrieve the requested product details by invoking the appropriate product information function. Provide the customer with product name, price, availability, features, warranty, and a recommendation.

Constraints:

- Return only verified product information.
- If the product does not exist, return an appropriate error message.
- If the customer does not enter a product name, request a valid product name.
- Do not generate fictional information.

Output Format:

Return the response in structured JSON format.

How the Improved Prompt Solves the Problems

The redesigned prompt improves the overall performance of the AI assistant in several ways.

Role Definition

Assigning the AI the role of a Product Information Assistant ensures consistent and domain-specific responses.

Context

Providing context allows the AI to understand the application scenario and generate more relevant answers.

Task Instructions

Clearly defining the task ensures that only relevant product information is returned.

Constraints

Constraints prevent the model from producing misleading or fabricated responses while ensuring validation of incorrect inputs.

Structured Output

Using JSON ensures that the output is organized, machine-readable, and suitable for integration into software systems.

Error Handling

The improved prompt instructs the AI to respond appropriately when invalid or unavailable products are requested.

Consistency

The redesigned prompt produces reliable and standardized responses across multiple executions.

Technologies Used

The proposed system uses Python-based technologies to simulate an AI-powered Product Information Assistant implementing Function Calling and Structured Output. The selected technologies are lightweight, easy to understand, and suitable for demonstrating the workflow without requiring external APIs.

Technology	Purpose	Justification
Python 3.x	Core programming language	Python provides simple syntax, extensive libraries, and is widely used for AI and automation applications.
VS Code	Development Environment	Provides an efficient environment for coding, debugging, and execution of Python programs.
JSON	Structured Output Format	Ensures consistent, machine-readable, and standardized responses that can easily integrate with other systems.
Python Dictionary	Product Database	Used to simulate a product database for retrieving product information without requiring an external database.
Function Calling	Invoking predefined functions	Enables the AI system to execute appropriate functions instead of generating random responses.

Techniques Used

The implementation follows Prompt Engineering and Function Calling techniques to improve response quality and system reliability.

Technique	Purpose	Benefit
Role Prompting	Defines the AI as a Product Information Assistant	Produces consistent and domain-specific responses
Context Prompting	Provides shopping scenario	Improves relevance of generated responses
Task Prompting	Clearly specifies required task	Reduces ambiguity and irrelevant information
Constraint Prompting	Restricts invalid outputs	Prevents hallucination and improves accuracy
Structured Output	Returns JSON response	Easy integration with software applications
Function Calling	Executes Python functions	Retrieves reliable product information from database
Input Validation	Checks user inputs	Prevents invalid function execution

Function Calling Schema

Purpose

The Function Calling Schema defines how the AI assistant invokes the appropriate function to retrieve product information based on the user's request.

Function Name

get_product_details(product_name)

Function Description

Retrieves product details from the product database using the product name provided by the user.

Function Parameters

Parameter	Data Type	Required	Description
product_name	String	Yes	Name of the product entered by the customer

Return Type

JSON Object

Returned Fields

Field	Data Type	Description
product_name	String	Name of the product
price	Integer	Product price
availability	String	Product availability
features	Array	List of product features
warranty	String	Warranty information
recommendation	String	AI recommendation
error	String	Error message (if applicable)

Structured Output (JSON Schema)

The system returns responses in a predefined JSON format to ensure consistency and interoperability.

```
{
  "product_name": "string",
  "price": "integer",
  "availability": "string",
  "features": [
    "string"
  ],
  "warranty": "string",
  "recommendation": "string"
}
```

Error Response Schema

If the requested product is unavailable:

```
{
  "error": "Product not found."
```

}

If the product name is missing:

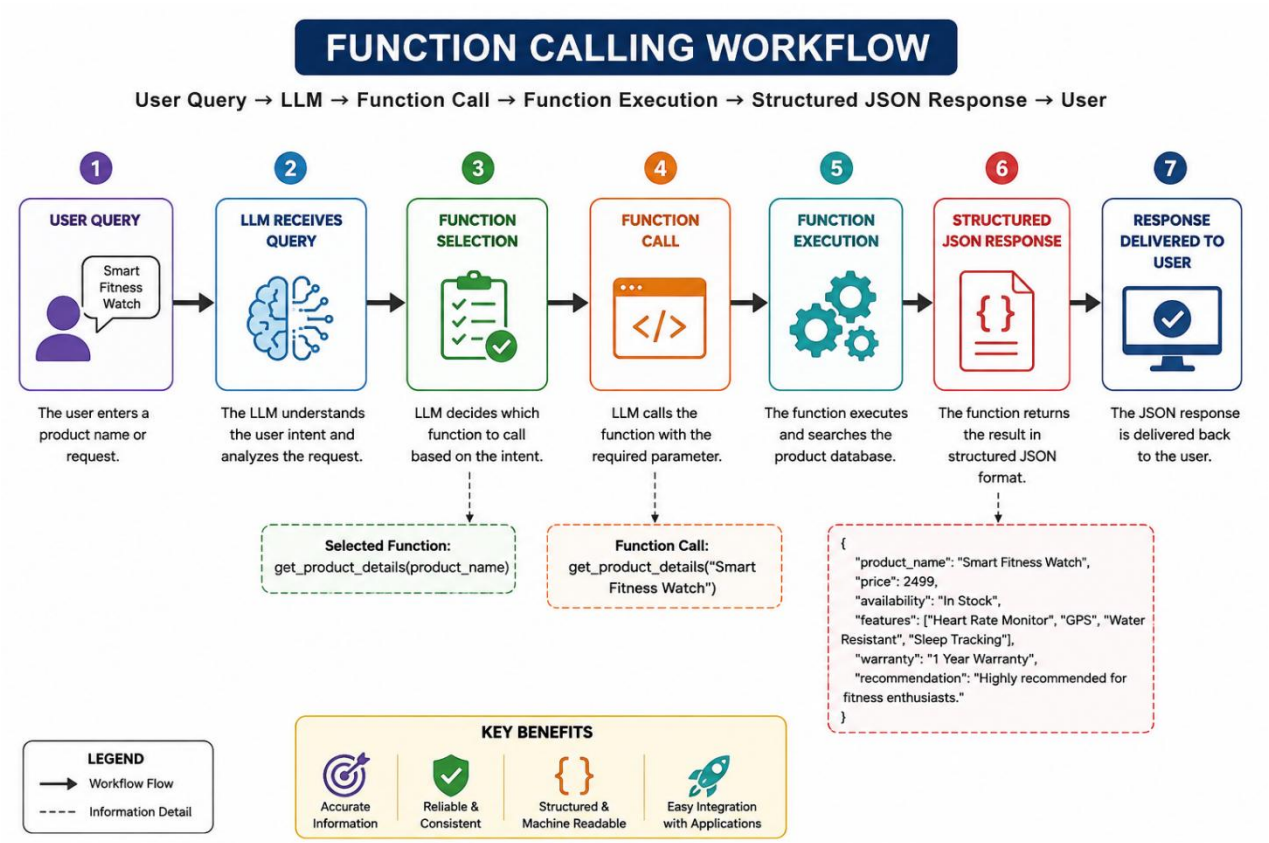
{

"error": "Please enter product name."

}

Function Calling Workflow

The implemented system follows the Function Calling workflow shown below.



Workflow Explanation

Step 1: The customer enters a product name.

Step 2: The LLM analyzes the user's request and identifies that the customer is asking for product information.

Step 3: Based on the identified intent, the LLM selects the predefined function get_product_details().

Step 4: The function searches the simulated product database.

Step 5: If the product exists, the function retrieves its details and formats them as structured JSON.

Step 6: If the product does not exist or the input is empty, an appropriate error message is returned in JSON format.

Step 7: The structured response is displayed to the user, completing the Function Calling workflow.

Source Code

Implementation

The complete implementation was developed in Python using Visual Studio Code. The program simulates an AI Product Information Assistant capable of performing Function Calling and generating Structured JSON Output.

The implementation consists of the following components:

- Product database
- Function for retrieving product information
- LLM workflow simulation
- Function selection
- JSON response generation
- Error handling
- Test case execution

Code:

```
import json

# ==========================================================

# PRODUCT DATABASE

# ==========================================================

products = {

    "smart fitness watch": {

        "product_name": "Smart Fitness Watch",

        "price": 4999,

        "availability": "In Stock",
```

```
    "features": [
      "Heart Rate Monitor",
      "Sleep Tracking",
      "GPS",
      "Water Resistant"
    ],
    "warranty": "1 Year",
    "recommendation": "Recommended for fitness enthusiasts."
  },
  "wireless earbuds": {
    "product_name": "Wireless Earbuds",
    "price": 2999,
    "availability": "In Stock",
    "features": [
      "Noise Cancellation",
      "Bluetooth 5.3",
      "20-Hour Battery",
      "Fast Charging"
    ],
    "warranty": "6 Months",
    "recommendation": "Best for music lovers and travelers."
  },
  "gaming mouse": {
    "product_name": "Gaming Mouse",
    "price": 1999,
```

```
        "availability": "Limited Stock",
        "features": [
            "RGB Lighting",
            "16000 DPI",
            "Programmable Buttons",
            "Ergonomic Design"
        ],
        "warranty": "2 Years",
        "recommendation": "Suitable for gamers."
    }
}
# ==========================================================
# FUNCTION CALL
# ==========================================================
def get_product_details(product_name):
    """
    Retrieves product details from database.
    """
    product_name = product_name.strip().lower()
    if product_name == "":
        return {
            "error": "Please enter product name."
        }
    if product_name in products:
        return products[product_name]
```

```
    return {
        "error": "Product not found."
    }
# ========================================================
# LLM SIMULATION
# ========================================================
def llm_assistant(user_query):
    print("\n")
    print("="*70)
    print("FUNCTION CALLING WORKFLOW")
    print("="*70)
    print("\nStep 1 : User Query")
    print("-------------------------------------------------")
    print(user_query if user_query.strip() else "[Empty Input]")
    print("\nStep 2 : LLM Analysis")
    print("-------------------------------------------------")
    print("The LLM analyzes the user's request.")
    print("Intent detected : Product Information Request")
    print("\nStep 3 : Function Selection")
    print("-------------------------------------------------")
    print("Selected Function : get_product_details(product_name)")
    print("\nStep 4 : Function Execution")
    print("-------------------------------------------------")
    response = get_product_details(user_query)
    if "error" in response:
```

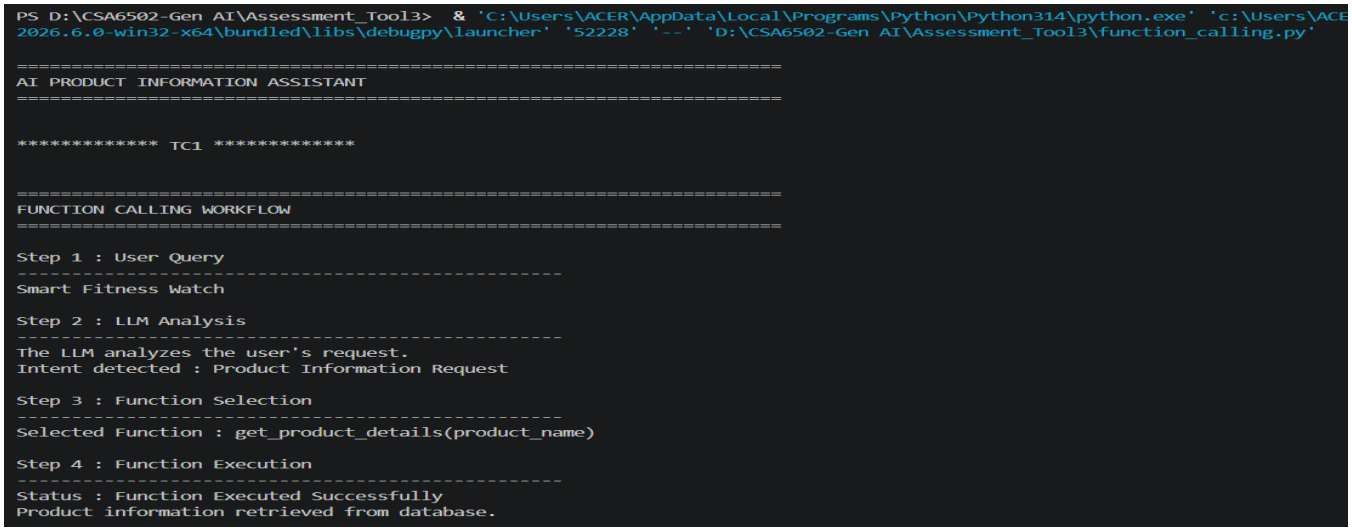
```
        print("Status : Function Executed")
        print("Result : Error handled successfully.")
    else:
        print("Status : Function Executed Successfully")
        print("Product information retrieved from database.")
    print("\nStep 5 : Structured JSON Response")
    print("-------------------------------------------------")
    print(json.dumps(response, indent=4))
    print("\nStep 6 : Response Delivered to User")
    print("-------------------------------------------------")
    print("Response generated successfully.")
    print("="*70)
    return response
    # ========================================================
# TEST CASES
# ========================================================
print("\n")
print("="*70)
print("AI PRODUCT INFORMATION ASSISTANT")
print("="*70)
test_cases = [
    ("TC1", "Smart Fitness Watch"),
    ("TC2", "Wireless Earbuds"),
    ("TC3", "Gaming Mouse"),
    ("TC4", "Laptop"),
```

```
    ("TC5", "")
]

for tc, query in test_cases:
    print(f"\n\n************* {tc} *************")
    llm_assistant(query)
```

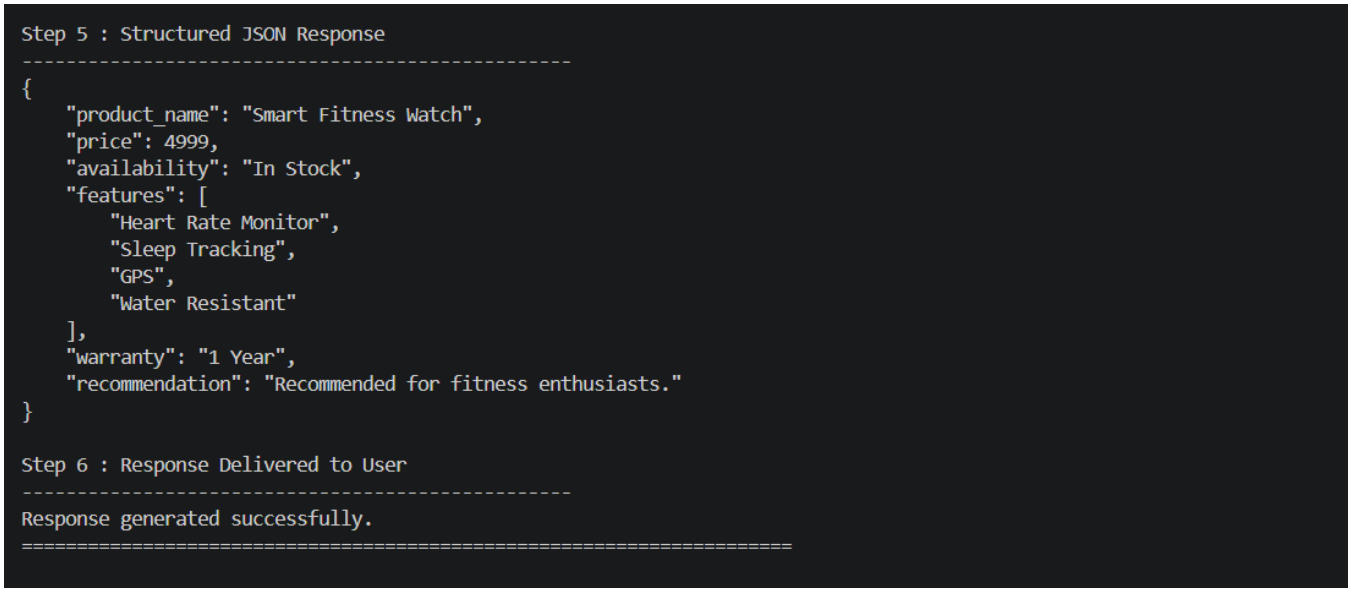
Test Cases

Figure 1 – Test Case 1: Smart Fitness Watch

Screenshot:



```
PS D:\CSA6502-Gen AI\Assessment_Tool3>  & 'C:\Users\ACER\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\ACE
2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52228' '--' 'D:\CSA6502-Gen AI\Assessment_Tool3\function_calling.py'

======================================================================
AI PRODUCT INFORMATION ASSISTANT
======================================================================


************* TC1 *************


======================================================================
FUNCTION CALLING WORKFLOW
======================================================================

Step 1 : User Query
--------------------------------------------------
Smart Fitness Watch

Step 2 : LLM Analysis
--------------------------------------------------
The LLM analyzes the user's request.
Intent detected : Product Information Request

Step 3 : Function Selection
--------------------------------------------------
Selected Function : get_product_details(product_name)

Step 4 : Function Execution
--------------------------------------------------
Status : Function Executed Successfully
Product information retrieved from database.
```



```
Step 5 : Structured JSON Response
--------------------------------------------------
{
    "product_name": "Smart Fitness Watch",
    "price": 4999,
    "availability": "In Stock",
    "features": [
        "Heart Rate Monitor",
        "Sleep Tracking",
        "GPS",
        "Water Resistant"
    ],
    "warranty": "1 Year",
    "recommendation": "Recommended for fitness enthusiasts."
}

Step 6 : Response Delivered to User
--------------------------------------------------
Response generated successfully.
======================================================================
```

Description:

The user requested information about the **Smart Fitness Watch**. The LLM analyzed the request, selected the get_product_details() function, retrieved the product information from the database, and returned a structured JSON response containing the product name, price, availability, features, warranty, and recommendation.

Figure 2 – Test Case 2: Wireless Earbuds

Screenshot:

```
************* TC2 *************


======================================================================
FUNCTION CALLING WORKFLOW
======================================================================

Step 1 : User Query
--------------------------------------------------
Wireless Earbuds

Step 2 : LLM Analysis
--------------------------------------------------
The LLM analyzes the user's request.
Intent detected : Product Information Request

Step 3 : Function Selection
--------------------------------------------------
Selected Function : get_product_details(product_name)

Step 4 : Function Execution
--------------------------------------------------
Status : Function Executed Successfully
Product information retrieved from database.
```

```
Step 5 : Structured JSON Response
--------------------------------------------------
{
    "product_name": "Wireless Earbuds",
    "price": 2999,
    "availability": "In Stock",
    "features": [
        "Noise Cancellation",
        "Bluetooth 5.3",
        "20-Hour Battery",
        "Fast Charging"
    ],
    "warranty": "6 Months",
    "recommendation": "Best for music lovers and travelers."
}

Step 6 : Response Delivered to User
--------------------------------------------------
Response generated successfully.
======================================================================
```

Description:

The user requested details of **Wireless Earbuds**. The system successfully executed the function call and displayed the product information in the required JSON format.

Figure 3 – Test Case 3: Gaming Mouse

Screenshot:

```
********** TC3 **********


======================================================================
FUNCTION CALLING WORKFLOW
======================================================================

Step 1 : User Query
--------------------------------------------------
Gaming Mouse

Step 2 : LLM Analysis
--------------------------------------------------
The LLM analyzes the user's request.
Intent detected : Product Information Request

Step 3 : Function Selection
--------------------------------------------------
Selected Function : get_product_details(product_name)

Step 4 : Function Execution
--------------------------------------------------
Status : Function Executed Successfully
Product information retrieved from database.
```

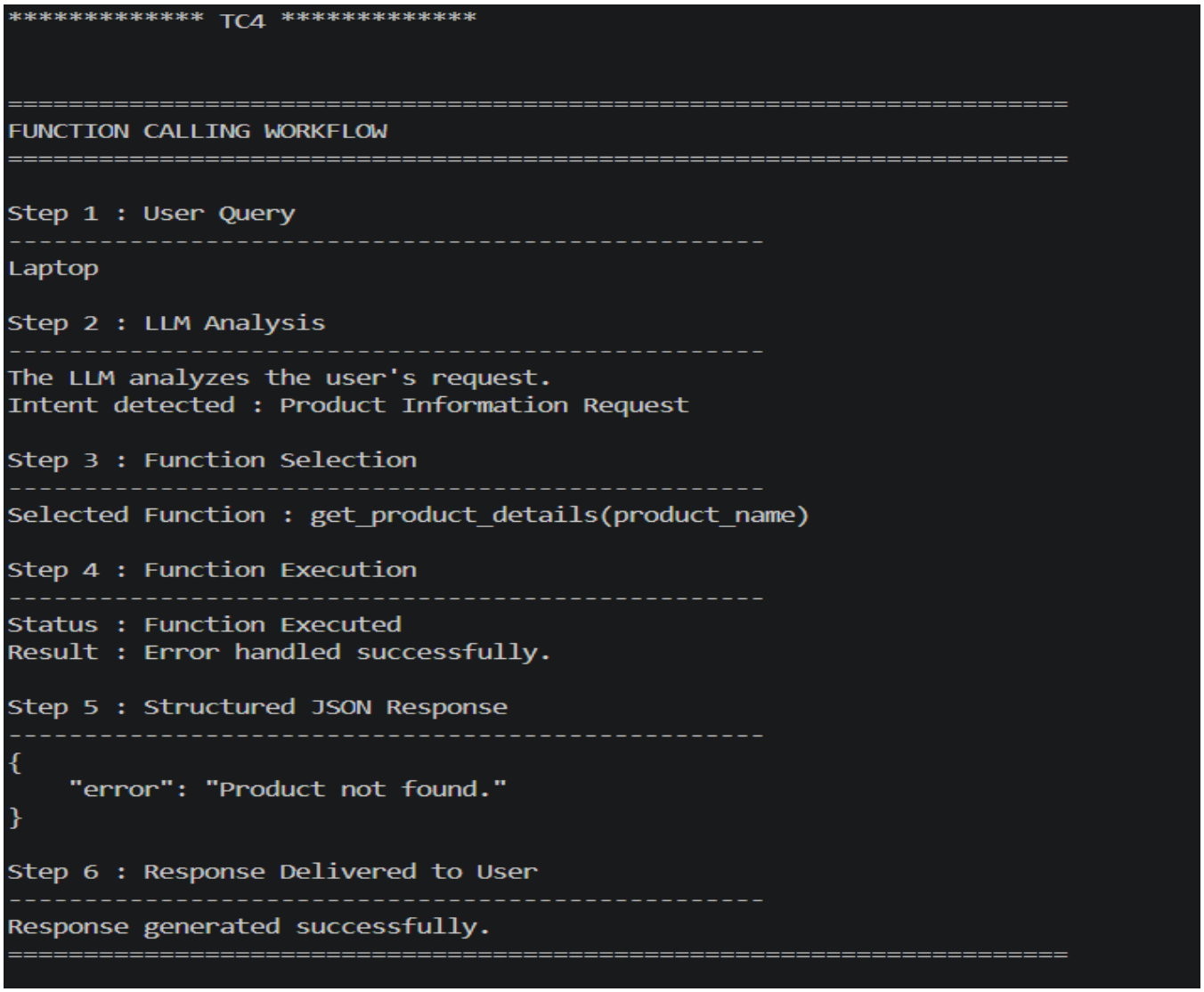
```
Step 5 : Structured JSON Response
--------------------------------------------------
{
    "product_name": "Gaming Mouse",
    "price": 1999,
    "availability": "Limited Stock",
    "features": [
        "RGB Lighting",
        "16000 DPI",
        "Programmable Buttons",
        "Ergonomic Design"
    ],
    "warranty": "2 Years",
    "recommendation": "Suitable for gamers."
}

Step 6 : Response Delivered to User
--------------------------------------------------
Response generated successfully.
======================================================================
```

Description:

The system successfully processed the request for **Gaming Mouse** and generated a structured JSON response containing all available product details.

Figure 4 – Test Case 4: Product Not Found (Edge Case)

Screenshot:



Description:

The user entered **Laptop**, which is not available in the product database. The system correctly detected that the requested product does not exist and returned the following structured error response:

{

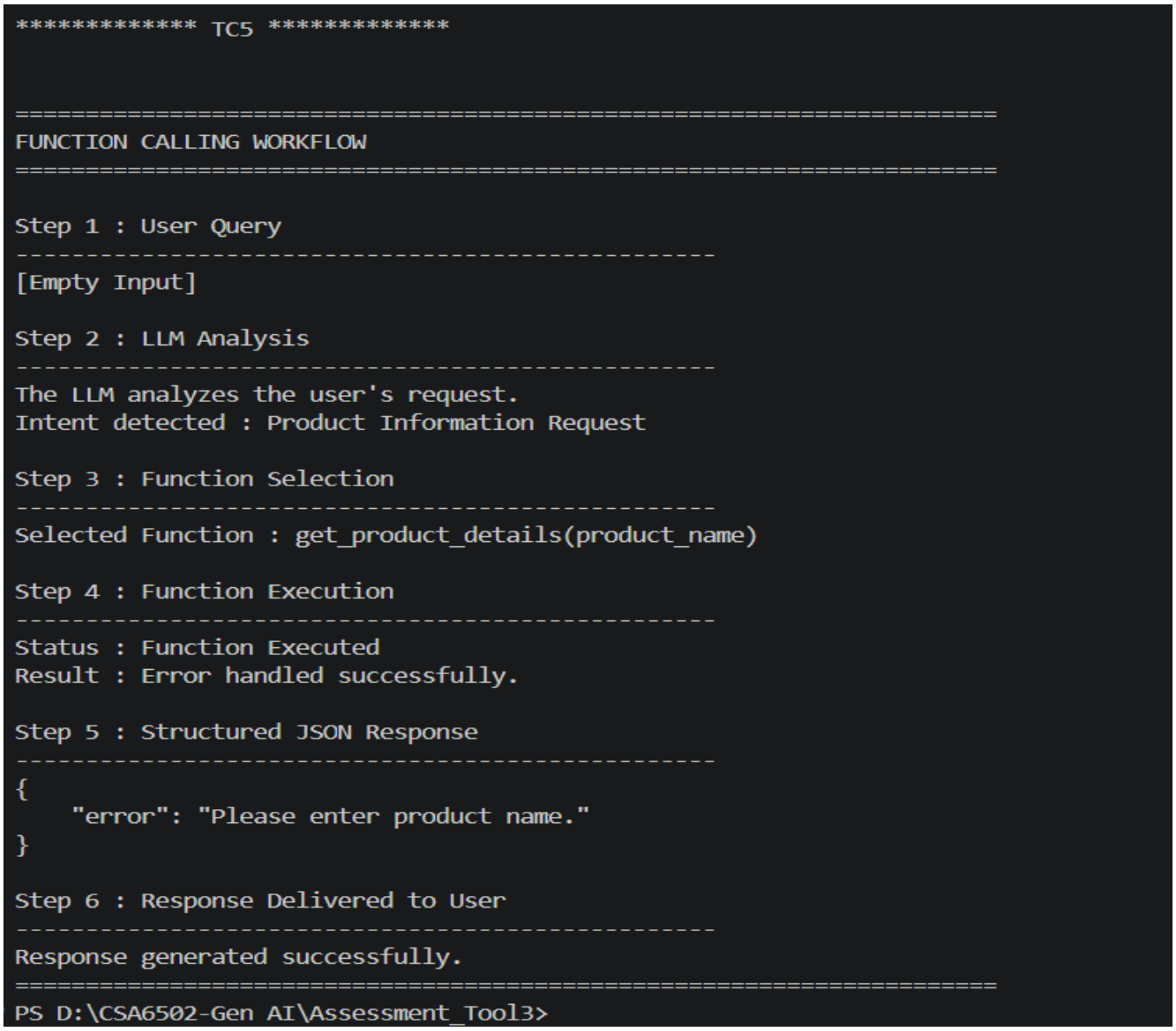
"error": "Product not found."

}

This demonstrates proper handling of unavailable information.

Figure 5 – Test Case 5: Empty Input (Edge Case)

Screenshot:



Description:

The user submitted an empty product name. The system validated the input before attempting the function call and returned the following structured error response:

{

"error": "Please enter product name."

}

This demonstrates proper validation of missing parameters.

Test Cases and Results

Test Case	Input	Expected Result	Actual Result	Status
TC1	Smart Fitness Watch	Product details returned in JSON	Product details returned in JSON	Pass
TC2	Wireless Earbuds	Product details returned in JSON	Product details returned in JSON	Pass
TC3	Gaming Mouse	Product details returned in JSON	Product details returned in JSON	Pass
TC4	Laptop	Error: Product not found	Error: Product not found	Pass
TC5	Empty Input	Error: Please enter product name	Error: Please enter product name	Pass

Brief Documentation

Purpose

The objective of the developed system is to demonstrate Function Calling and Structured Output concepts using Python. Instead of allowing the AI to generate arbitrary responses, the system retrieves verified product information through predefined functions.

Modules

Product Database Module

Stores product information such as product name, price, availability, features, warranty, and recommendations using Python dictionaries.

Function Calling Module

Implements the get_product_details() function, which retrieves product information based on the user query.

LLM Simulation Module

Simulates the reasoning process of a Large Language Model by analyzing the user query, selecting the appropriate function, and executing it.

Structured Output Module

Converts the retrieved product information into structured JSON format for consistent presentation.

Validation Module

Handles invalid inputs such as empty product names and unavailable products by generating appropriate JSON error responses.

Working Principle

1. The user enters a product name.
2. The LLM analyzes the request.
3. The LLM selects the get_product_details() function.
4. The function searches the product database.
5. If the product exists, its details are returned in JSON format.
6. If the product is unavailable or the input is invalid, an appropriate error message is returned.
7. The response is displayed to the user.

Benefits

- Provides accurate and consistent product information through predefined functions.
- Produces structured JSON output that can be easily integrated with applications and APIs.
- Handles invalid inputs and unavailable products gracefully through validation.
- Demonstrates a clear Function Calling workflow suitable for AI-based applications.
- Easy to maintain, extend, and integrate with larger systems or real databases.

Limitations

- Uses a small simulated product database instead of a real database.
- Does not connect to external APIs or live product inventories.
- Supports only predefined products.
- The LLM reasoning is simulated and does not use an actual cloud-based Large Language Model.
- The application is intended for educational demonstration and not for production deployment.

Conclusion

This assessment successfully demonstrated the implementation of Function Calling and Structured Output using Python. The developed AI Product Information Assistant receives customer queries, analyzes user intent, invokes the appropriate function, retrieves verified product information from a simulated database, and returns the results in structured JSON format. The implementation also includes validation mechanisms for missing inputs and unavailable products, ensuring reliable and consistent responses.

The use of Function Calling improves the accuracy of generated responses by relying on predefined functions rather than unrestricted text generation. Structured JSON output enhances interoperability, making the system suitable for integration with other software applications. Overall, the project illustrates how Prompt Engineering, Function Calling, and Structured Output can be combined to build reliable AI-assisted applications.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Schema Design	30%	Correctly designs a function-calling schema and structured (JSON) output format	Mostly correct schema with minor issues	Schema has significant errors	Schema is fundamentally incorrect or missing
Output Validity	25%	LLM reliably produces valid structured output matching the schema	Mostly valid output with occasional errors	Output is frequently invalid or inconsistent	Output rarely or never matches the schema
Edge-Case Handling	20%	Handles edge cases (missing fields, ambiguous input) gracefully	Handles common cases well	Handles only the simplest cases	Fails on basic cases
Function-Call Flow Demo	15%	Clear demo of the function-call to structured-response flow	Demo covers the main flow	Demo is incomplete	No functional demo presented
Schema Documentation	10%	Well-documented schema design rationale	Adequate documentation	Minimal documentation	No documentation provided

Marks Evaluation:

Question No.	Schema Design(30%)	Output Validity (25%)	Edge-Case Handling (20%)	Function-Call Flow Demo(15%)	Schema Documentation (10%)	Total Marks (100)
Q1						
					Overall Total (100)	

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____